
task-app 30日間カリキュラム

Day 22

グラフを表示しよう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	3
今日のゴール	3
始める前の前提	4
なぜこれを作るのか	5
実装ステップ一覧	9
Step 1: Rechartsを確認する(読む目安: 3分)	11
Step 2: インポートと定数を追加する(読む目安: 4分)	12
Step 3: ステータス表示データを作る(読む目安: 5分)	16
Step 4: ステータス円グラフのCard枠を作る(読む目安: 5分)	18
Step 5: Pieにデータとセル色を設定する(読む目安: 5分)	20
Step 6: 優先度表示データを作る(読む目安: 4分)	23
Step 7: 優先度円グラフを追加する(読む目安: 5分)	25
Step 8: グリッドに配置して完成(読む目安: 4分)	29
Step 9: 動作確認(読む目安: 3分)	30
Pro パターンで書こう(集計はクライアントかサーバーか)	32
完成コード全体	34
今日のまとめ	51
つまずきポイント	52
今日学んだ用語	52
理解チェック	53
追加課題:円グラフの大きさだけを変える	54
次回予告	55
次に読むもの	55
奥付	55

前回の振り返り

Day 21 ではサーバー側で集計済みのデータを返す `api.report.getOverview` を呼び出しました。タスク数・完了率・合計作業時間・平均作業時間の 4 枚の統計カードを表示しています。数値をカードで見せる基盤ができたので今日は同じ集計済みデータを円グラフで可視化します。

今日のゴール

Recharts(React 用のグラフ描画ライブラリ)を使ってレポートページに円グラフを追加します。`getOverview` が返す集計済みデータからステータス別・優先度別のタスク分布を表示します。

スクリーンショット: レポートページにステータス別・優先度別の円グラフが並んだ完成イメージです。

Day 22: グラフを表示しよう



図1: レポートページ。統計カード4枚の下に、ステータス別タスクと優先度別タスクの円グラフが横に2枚並んでいる

今日のゴールライン: サーバーが集計したタスク分布を受け取り、Recharts の部品を組み合わせて円グラフとして画面に出せればOKです。

始める前の前提

- Day 21 のレポートページと統計カードが表示できます
- `getOverview` が集計済みデータを返すことを Day 21 で確認済み
- タスクが複数件あり、ステータスや優先度にばらつきがあります
- `recharts` がインストール済みであることを確認できます

3つ目の前提は初期データのままで満足せません。自分に見えるタスクは「Webサイトリニューアル」の3件で、ステータスは未対応・進行中・完了が1件ずつ、優先度は高が2件と中が1件です。低と緊急は1件もありません。このまま開

くとステータスの円は同じ大きさの3切れ、優先度の円は2切れになります。今日の主題は偏りを目で見ることなので `/task` で優先度と状態を散らしたタスクを4~5件足してから始めると扇の数と大きさの違いが画面で確かめられます。足さないまま進んでも、コードの書き方は変わりません。

なぜこれを作るのか

統計カードは合計や割合は分かりますがどのステータスにタスクが偏っているかまでは読み取れません。円グラフにするとその偏りをひと目で把握できます。

例え話: 円グラフは「天気予報の図」です。「降水確率60%」と聞くより雨雲の図を見た方が直感的に伝わります。数値を円グラフにすると各ステータスの割合が視覚的に伝わります。

なぜサーバー側で集計するのか

円グラフに必要なのは「ステータスごとの件数」だけです。全タスクを丸ごとクライアントへ送ると件数が増えるほど通信量が膨らみます。サーバーで件数まで数えておけば送るデータは数行の集計結果で済みます。ブラウザは受け取った配列をそのまま描くので表示も速くなります。

Day 21 で見た通り、この集計はすでに `getOverview` の中で終わっています。今日のフロントの仕事は「受け取った集計結果に日本語ラベルと色を足して描く」ことです。

グラフ表示のデータフロー

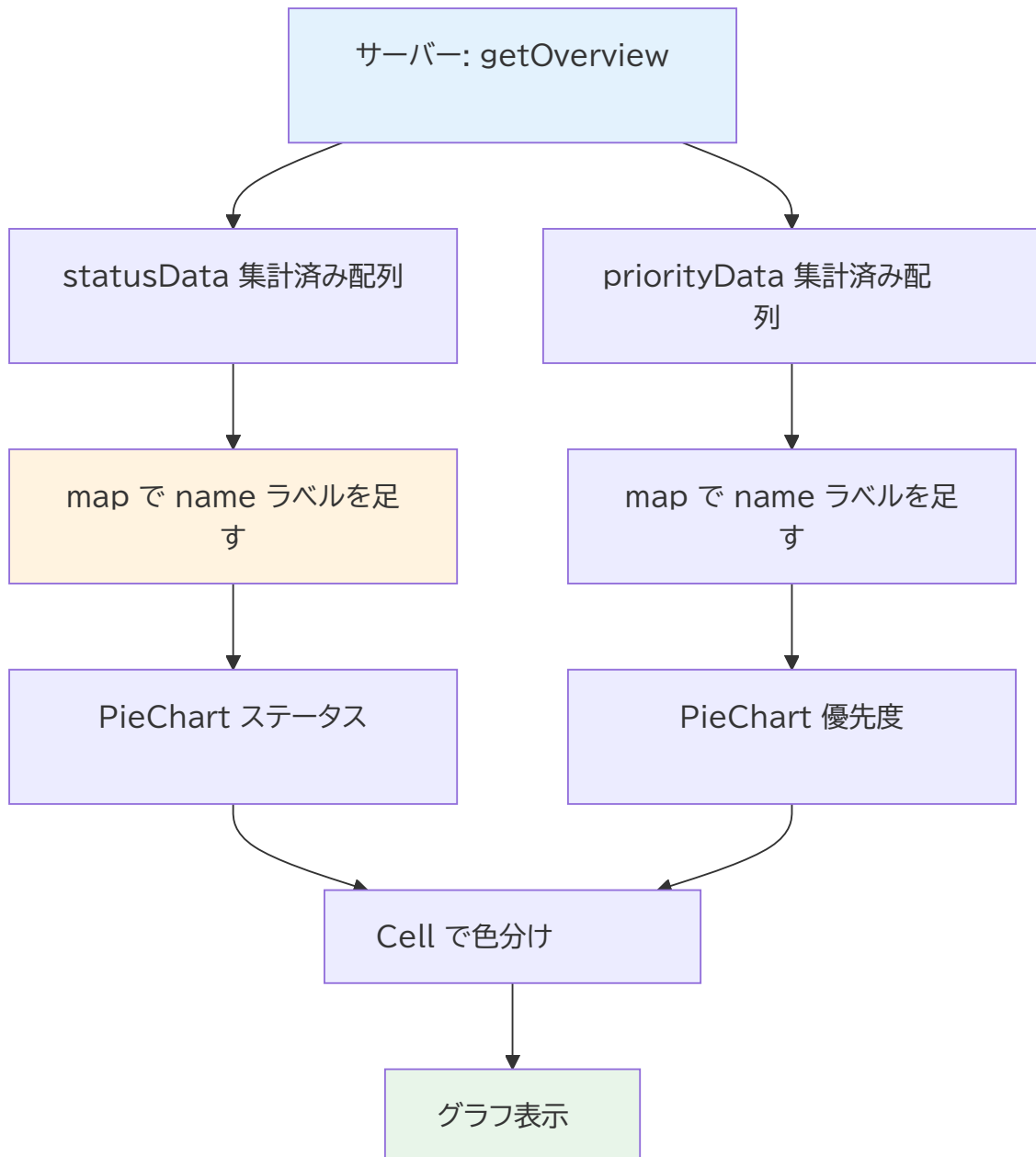


図2: グラフ表示のデータフロー

図の上半分がサーバー、下半分がブラウザの担当です。`getOverview` から出てくる `statusData` と `priorityData` はこの時点ですでに件数まで数え終わった配列になっています。ブラウザ側の枝で起きるのは `map` で日本語ラベルを足すところだけで、そこから先は `PieChart` へ渡して `Cell` で色を付けるだけです。件数

を数える処理がブラウザ側の枝に1つも出てこない点を、先に頭へ入れておいてください。今日のコードで `filter` や `length` が出てこないのは書き忘れではなくこの分担のためです。

やること / やらないこと

やること	やらないこと
ステータス別円グラフ	棒グラフ
優先度別円グラフ	折れ線グラフ
集計済みデータの表示	クライアント側での再集計
色分け・レスポンシブ対応	アニメーション

新しく学ぶ概念

概念	読み方	役割	例え
Recharts	リチャーツ	React 用のグラフ描画ライブラリ	画用紙とペンのセット
PieChart	パイチャート	円グラフの枠組み	ピザを載せるお皿
Cell	セル	各スライスに色を付ける部品	ピザの各ピースに振る色
dataKey / nameKey	データキー / ネームキー	データのどのプロパティを使うか指定	名簿のどの列を読むかの指定
Responsive Container	—	グラフのサイズを親に合わせる	額縁に合わせるキャンバス

サーバー集計済みデータの形

`getOverview` が返す `statusData` は `key` (ステータス名) と `value` (件数) だけを持つ配列です。日本語ラベルの `name` と色はフロント側で `key` から引いて足します。

段階	データの形と例
サーバーが返す	形: <code>{ key, value }</code> 例: <code>{ key: 'TODO', value: 3 }</code>
フロントで <code>name</code> を足す	形: <code>{ key, value, name }</code> 例: <code>{ key: 'TODO', value: 3, name: '未対応' }</code>
色は <code>key</code> から引く	形: <code>TASK_STATUS_COLORS[key]</code> 例: <code>'#5f6777'</code>

`key` は `'TODO'` のような英字のステータス名です。人が読む見出しには使いにくいので `name` に日本語ラベルを入れて凡例やツールチップに出します。

Recharts の組み立てパターン

Recharts は複数のコンポーネント(画面を組み立てる部品)を組み合わせます。「枠組み + 中身 + 補助部品」の3層で1つのグラフを構成します。

層	コンポーネント	役割
枠組み	<code>PieChart</code>	グラフ全体の座標空間

層	コンポーネント	役割
中身	<code>Pie</code> + <code>Cell</code>	データを描画し色を付ける
補助	<code>Tooltip</code> / <code>Legend</code>	ホバー表示と凡例

Day 23 では `BarChart` や `LineChart` も登場しますが3層構造は共通です。

開発サーバーは前の Day から動かしたまま使います。止めてあるときは `npm run dev` で起動してから `http://localhost:3000` を開きます。

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 1	Rechartsを確認する	3分
Step 2	インポートと定数を追加する	4分
Step 3	ステータス表示データを作る	5分
Step 4	ステータス円グラフのCard枠を作る	5分
Step 5	Pieにデータとセル色を設定する	5分
Step 6	優先度表示データを作る	4分

ステップ	作業内容	読む時間の目安
Step 7	優先度円グラフを追加する	5 分
Step 8	グリッドに配置して完成	4 分
Step 9	動作確認	3 分

読む時間の合計(仮): 約 38 分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

予備知識: 今日使う Recharts コンポーネント

コンポーネント	役割	例え
<code>PieChart</code>	円グラフ全体の枠組み	パイを描く土台
<code>Pie</code> + <code>Cell</code>	各スライスの色を <code>Cell</code> で設定	パイの各ピース
<code>ResponsiveContainer</code>	グラフを親要素の幅に合わせる	額縁サイズの自動調整
<code>Tooltip</code>	マウスホバーで数値を表示	ポイントの拡大表示
<code>Legend</code>	凡例(色と名前の対応表)	地図の凡例

Step 1: Rechartsを確認する(読む目安: 3分)

ゴール: Recharts がすでにインストール済みであることを確認します。

次のコマンドで確認します。

```
# filepath: ターミナル(確認のみ)
# Recharts がインストール済みか確認する
npm list recharts
# recharts@3.x.x が表示されればOK
```

Recharts は React 用のグラフライブラリで、`PieChart` や `BarChart` など宣言的にグラフを描けます。このプロジェクトでは Day 01 で実行した `scripts/scaffold-from-scratch.sh` が入れているのであらためて入れる必要はありません。

バージョンではなく `(empty)` と表示された場合は、Recharts が入っていません。Day 01 のスクリプトを途中で止めたときなどに起こります。次のコマンドで、Day 01 と同じ3系を入れてから Step 2 へ進んでください。

```
# filepath: ターミナル((empty) と出たときだけ)
# Day 01 と同じ Recharts 3系を入れる
npm install recharts@^3.2.1
```

入っていないまま Step 2 以降を写経すると、import の解決時に

`Module not found: Can't resolve 'recharts'` が出ます。先にインストールしてから画面を開き直します。

Day 22: グラフを表示しよう

確認ポイント:

- recharts が package.json にあります
- バージョンが `3.x.x` と表示されました

Step 2: インポートと定数を追加する(読む目安: 4分)

ゴール: Day 21 のコードに Recharts と色定数のインポートを追加します。

Day 21 では `Card` , `CardContent` , `CardHeader` , `CardTitle` をすでにインポートしています。今回はそこに Recharts と色・ラベルの定数を追加します。

実装: Day 21 のインポート部分に、以下の 3 ブロックを追加してください。

```
// filepath: src/app/report/page.tsx
// Rechartsのグラフ描画コンポーネントを追加
import {
  Cell, Legend, Pie, PieChart,
  ResponsiveContainer, Tooltip,
} from 'recharts';
```

6つをまとめて取り込みますが役割は3つに分かれます。`PieChart` は扇を並べる座標の土台、`Pie` と `Cell` は扇そのものと1切れずつの色、`Tooltip` と `Legend` は読み手向けの補助表示です。`ResponsiveContainer` だけは何も描かず、親要素の幅と高さをグラフへ伝える係を務めます。取り込みを1つでも落とす

とその部品を書いた行で「定義されていない名前を使っている」というエラーになり、開発中はエラー画面が表示されます。Day 21 で `Card` を取り込んだときと同じ書き方なので既存の `import` の並びに足すだけで済みます。

Recharts の取り込みで、ブラウザへ送る描画用のコードが増えます。Day 21 の Server/Client 分離は送るコードの範囲を決める方法です。読み込みを後回しにするには別ファイルへ分けたうえで `next/dynamic` などの遅延読み込みを設定します。今日は通常の `import` でグラフ表示を確かめます。遅延読み込みの実装は扱いません。

確認ポイント:

- Recharts のインポートが追加されました

```
// filepath: src/app/report/page.tsx
// 優先度の色・ラベルと型ガード関数を追加
import {
  isTaskPriority,
  TASK_PRIORITY_COLORS,
  TASK_PRIORITY_LABELS,
} from '@lib/constant/priority';
```

ここで取り込む3つは優先度の `key` を人が読める形へ変換するための道具です。

`TASK_PRIORITY_LABELS` は `'HIGH'` から `'高'` を、

`TASK_PRIORITY_COLORS` は同じ `'HIGH'` から扇の色を引く対応表になっています。

`isTaskPriority` は届いた値が優先度の4種類のどれかを実行時に確かめる関数(型ガード)です。型の上では `entry.key` は Prisma の enum と同じ4つ

Day 22: グラフを表示しよう

の文字列に決まっているため、この確認を外しても型エラーにはなりません。それでも確認するのは、実行中に対応表へ無い値が渡った場合に、代替色へ分岐できるようにするためです。

確認ポイント:

- `isTaskPriority` と色・ラベル定数をインポートしました

```
// filepath: src/app/report/page.tsx
// ステータスの色・ラベルと型ガード関数を追加
import {
  isTaskStatus,
  TASK_STATUS_COLORS,
  TASK_STATUS_LABELS,
} from '@/lib/constant/status';
```

優先度でそろえた3点セットを、今度はステータス側でも用意します。

`TASK_STATUS_LABELS` は `'TODO'` を `'未対応'` に、
`TASK_STATUS_COLORS` は `'TODO'` を `'#5f6777'` に対応づけた表です。どちらも `src/lib/constant/status.ts` にある定数で、Day 13 のタスク一覧で見出しを日本語にしたときと出どころは変わりません。ここで教材用の色をその場で書き起こすとあとで定数側だけ直したときに円グラフが古い色のまま取り残されます。1か所を直せば画面全体が追従する状態を保つために、定数から引きます。

確認ポイント:

- `isTaskStatus` と色・ラベル定数をインポートしました

```
// filepath: src/app/report/page.tsx
// key に対応する色がないときの代替色を定義
const CHART_FALLBACK_COLOR = '#9e9e9e';
```

色を引けず `undefined` を `Cell` に渡すと、その値が `Pie` の既定色を上書きし、扇のSVGには `fill` 属性が付きません。この教材のCSSにも扇の色を補う指定はないため、ブラウザの既定色である黒が表示されます。

`CHART_FALLBACK_COLOR` を明示すれば、対応表に無い値を黒ではなく `#9e9e9e` で表示できます。ステータスや優先度をあとから増やして定数の追加を忘れた場合も、ブラウザの既定値に任せません。優先度の「低」は `#5f6777` の濃いグレーで、代替色の `#9e9e9e` とは別の色です。グレー系どうしなので並ぶと似て見えることはあります。確かめたいときは優先度が「低」のタスクを1件作り、色の対応表と見比べてください。扇の数が想定と合わないときは色ではなくデータの中身を確認めます。

`CHART_FALLBACK_COLOR` は保険です。`key` が色の一覧にない値だったときだけ使います。通常は `TASK_STATUS_COLORS` から色が引けます。

確認ポイント:

- 上記4ブロックのインポートと定数を追加しました

ステータスの色一覧

ステータス	色	HEXコード
TODO	グレー	<code>#5f6777</code>
IN_PROGRESS	青緑	<code>#1e9cb8</code>

ステータス	色	HEXコード
IN_REVIEW	オレンジ	#f69e23
DONE	グリーン	#26ab7a
CANCELLED	レッド	#dc3848

この5色は `src/lib/constant/status.ts` の `TASK_STATUS_COLORS` に定義済みです。教材とアプリで同じ定数を使うので色がずれません。

Step 3: ステータス表示データを作る(読む目安: 5分)

ゴール: サーバーが返した集計済みの `statusData` に、日本語ラベルを足します。

Day 21 で使った `overview` (`getOverview` の戻り値)をそのまま使います。集計はサーバーで終わっているのでフロントでは数え直しません。

実装: Day 21 の `useQuery` の下、`return` 文の前に追加してください。

```
// filepath: src/app/report/page.tsx
// サーバーは key と value だけ返すので name をここで足す
const statusData =
  overview?.statusData.map((entry) => ({
    ...entry,
    name: isTaskStatus(entry.key)
      ? TASK_STATUS_LABELS[entry.key]
      : entry.key,
  }))) ?? [];
```

この配列の長さはサーバーが数えた分だけになります。Day 21 の

`activeTasksFilter` を思い出してください。あの絞り込みはアーカイブ済みプロジェクトのタスクと `CANCELLED` のタスクを集計から外しています。だから色の一覧に赤い `CANCELLED` が載っていてもその扇が円グラフに出ることはありません。中止したタスクを混ぜると「今動いている作業の内訳」として読めなくなるため、抜け落ちているのではなく意図して外してあります。統計カードの合計と円グラフの合計が一致するのも、両方が同じ絞り込みを通っているからです。

`overview?.statusData` の `?.` はデータがまだ届いていないときに `undefined` を返す書き方です。その場合は `?? []` で空配列にしてグラフが空でも落ちないようにしています。

確認ポイント:

- `overview.statusData` を `map` で加工しています
- `isTaskStatus` でラベルに変換しています
- クライアント側で件数を数え直していません

Day 22: グラフを表示しよう

- `npm run dev` でエラーが出ていません

この時点では画面に変化はありません。円グラフを描く `<Pie>` は Step 5 で追加するので凡例が日本語で並ぶのを確かめられるのはそのあとです。

なぜ map だけで済むのか

サーバーがすでに件数まで数えているのでフロントの仕事は `name` を1つ足すだけです。`entry.key` が `'TODO'` などのステータス名なら `TASK_STATUS_LABELS` から `'未対応'` を引きます。`statusData` の型は `tRPC`(型付き API 通信の仕組み)が推論するので自分で型を書く必要はありません。

Step 4: ステータス円グラフのCard枠を作る(読む目安: 5分)

ゴール: ステータス円グラフを表示する Card とグラフ枠を作ります。

以下のJSXは `return` 文の中、Day 21 の統計カード `</div>` の下に追加します。

実装:

```
{/* filepath: src/app/report/page.tsx */}
<Card>
  <CardHeader>
    <CardTitle>ステータス別タスク</CardTitle>
  </CardHeader>
  <CardContent>
    <div className="h-[300px]">
      <ResponsiveContainer width="100%" height="100%">
        <PieChart>
          {/* Step 5 で Pie を追加する */}
          <Tooltip />
          <Legend />
        </PieChart>
      </ResponsiveContainer>
    </div>
  </CardContent>
</Card>
```

`ResponsiveContainer` は幅と高さを `100%` で指定しています。この `100%` は親要素を基準にした割合です。親に高さがないと `0` と解釈され、グラフが表示されません。そのため親の `div` に `h-[300px]` で実際の高さを与えています。

確認ポイント:

- `Card` > `CardContent` > `div` > `ResponsiveContainer` の入れ子になっています
 - 親の `div` に `h-[300px]` があります
 - `PieChart` の中に `Tooltip` と `Legend` があります
-

Step 5: Pieにデータとセル色を設定する(読む目安: 5分)

ゴール: Step 4 の `{/* Step 5 で... */}` を `Pie` と `Cell` に置き換えます。

実装: Step 4 のコメント行を消して以下を `<PieChart>` の先頭に追加します。

```
{/* filepath: src/app/report/page.tsx */}
<Pie
  data={statusData}
  dataKey="value"
  nameKey="name"
  cx="50%"
  cy="50%"
  outerRadius={80}
  label
>
  {statusData.map((entry) => (
    <Cell
      key={entry.key}
      fill={
        isTaskStatus(entry.key)
          ? TASK_STATUS_COLORS[entry.key]
          : CHART_FALLBACK_COLOR
      }
    />
  ))}
</Pie>
```

`dataKey="value"` は各要素の `value` プロパティ(件数)を扇の大きさに使う指定です。`nameKey="name"` は `name` プロパティ(日本語ラベル)を凡例やツールチップの見出しに使います。データの形とこの名前が一致しないと扇が描かれず空のグラフになります。

Day 22: グラフを表示しよう

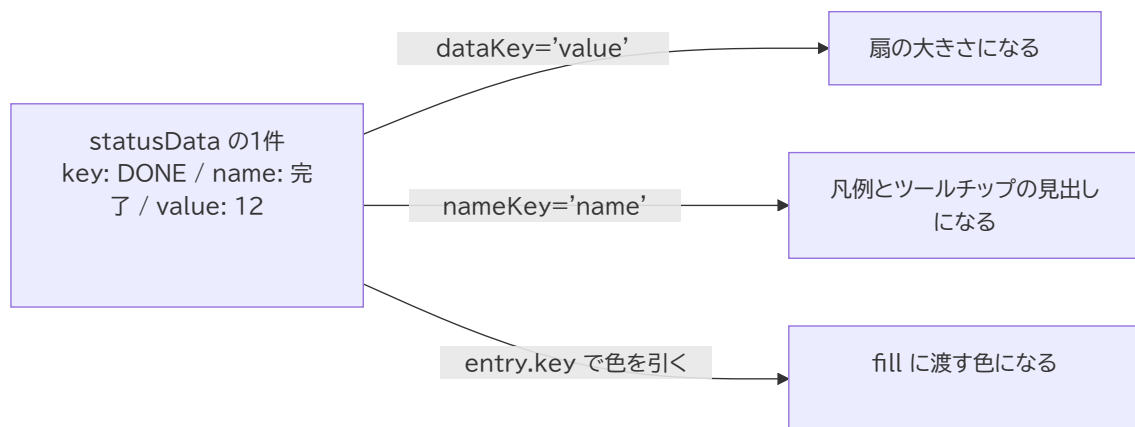


図 3: Step 5: Pie にデータとセル色を設定する

1 件のデータから 3 本の線が出てそれぞれ別の列を指します。`dataKey` を `key` に書き換えると `DONE` という文字を大きさに使おうとするので扇が描かれません。グラフが空のときはこの 3 本の行き先とデータの列名が合っているかを見てください。

`Cell` は `Pie` の子要素としてデータ 1 件ごとに 1 つ描画します。`entry.key` から `TASK_STATUS_COLORS` で色を引くので `isTaskStatus` 型ガードで安全に判定し、`as` 型アサーションは使いません。

`statusData` が空配列のときはこの `map` が 1 周も回りません。`Cell` は作られず、扇は 1 枚も描かれません。`Legend` は項目ゼロで何も出さず、カードの中には Step 4 で確保した 300px の空白だけが残ります。グラフが真っ白なときは Recharts の書き方を疑う前に、まず自分のプロジェクトにタスクが登録されているかを確認してください。参加しているプロジェクトが 1 つも無いアカウントではサーバーが最初から空配列を返します。

確認ポイント:

- `dataKey` と `nameKey` がデータのプロパティ名と一致しています
- `isTaskStatus` 型ガードで色を決定しています

- `as` 型アサーションを使っていません

画面を確認してください。この時点で表示される円グラフはステータス別の1つだけです。優先度別の円グラフは Step 7 で追加し、2つを横並びにするグリッド配置は Step 8 で作ります。

Step 4-5 の完成構造

Step 4 の Card 枠 + Step 5 の Pie を合わせると次のネスト構造になります。

階層	要素	由来
1	<code><Card></code>	Step 4
2	<code><CardHeader></code> + <code><CardContent></code>	Step 4
3	<code><ResponsiveContainer></code> > <code><PieChart></code>	Step 4
4	<code><Pie></code> > <code><Cell></code>	Step 5
4	<code><Tooltip /></code> + <code><Legend /></code>	Step 4

確認ポイント:

- Card 全体が1つのブロックとして完成しています

Step 6: 優先度表示データを作る(読む目安: 4分)

ゴール: サーバーが返した集計済みの `priorityData` に、日本語ラベルを足します。

Day 22: グラフを表示しよう

Step 3 の `statusData` と同じ場所(`return` 文の前)に追加してください。使う関数がステータス用から優先度用が変わるだけです。

実装:

```
// filepath: src/app/report/page.tsx
// priorityData も key と value だけなので name を足す
const priorityData =
  overview?.priorityData.map((entry) => ({
    ...entry,
    name: isTaskPriority(entry.key)
      ? TASK_PRIORITY_LABELS[entry.key]
      : entry.key,
  }))) ?? [];
```

ステータスと同じ手順です。型ガードが `isTaskPriority`、ラベルが `TASK_PRIORITY_LABELS` に変わる点だけが異なります。優先度もサーバーが集計済みなので数え直しはしません。

確認ポイント:

- `overview.priorityData` を `map` で加工しています
- `isTaskPriority` で型ガードしています
- Step 3 の `statusData` と同じ場所を書きました
- `npm run dev` でエラーが出ていません

ここでも画面はまだ変わりません。優先度の円グラフは Step 7 で追加します。

Step 7: 優先度円グラフを追加する(読む目安: 5分)

ゴール: 優先度別の円グラフをもう1枚追加します。

実装: Step 4-5 のステータスグラフの直後に、以下の Card を追加します。まずは Card 枠と **Pie** の設定です。

```
{/* filepath: src/app/report/page.tsx */}
<Card>
  <CardHeader>
    <CardTitle>優先度別タスク</CardTitle>
  </CardHeader>
  <CardContent>
    <div className="h-[300px]">
      <ResponsiveContainer width="100%" height="100%">
        <PieChart>
          <Pie
            data={priorityData}
            dataKey="value"
            nameKey="name"
            cx="50%"
            cy="50%"
            outerRadius={80}
            label
          />
        </PieChart>
      </div>
    </CardContent>
  </Card>
```

Day 22: グラフを表示しよう

枠の形は Step 4 のステータス側とそろえ、変わるのは `data` に渡す配列だけです。`cx` と `cy` の `50%` は円の中心を親要素の幅と高さのちょうど真ん中に置く指定です。`outerRadius={80}` は円の半径をピクセルで決めます。半径を親の高さの半分より大きくすると円が枠からはみ出して上下が切れるので `h-[300px]` の枠に対しては 80 前後が収まりのよい値になります。末尾の `label` は各扇のそばに件数を書き出す指定です。これを外すとホバーするまで件数が読めないグラフになります。

確認ポイント:

- Card 枠のネスト構造がステータスと同じ
- `data` が `priorityData` に変わっています

続けて `Cell` で色を付けて閉じます。

```

{ /* filepath: src/app/report/page.tsx */
  {priorityData.map((entry) => (
    <Cell
      key={entry.key}
      fill={
        isTaskPriority(entry.key)
          ? TASK_PRIORITY_COLORS[entry.key]
          : CHART_FALLBACK_COLOR
      }
    />
  ))}
</Pie>
<Tooltip />
<Legend />
</PieChart>
</ResponsiveContainer>
</div>
</CardContent>
</Card>

```

優先度の色は `TASK_PRIORITY_COLORS` から引きます。 `key` が `'HIGH'` などの優先度名なので `isTaskPriority` 型ガードで判定してから色を決めています。ここでも `as` は使いません。

確認ポイント:

- `isTaskPriority` 型ガードで色を決定しています

Day 22: グラフを表示しよう

- 2つの円グラフが表示されます

スクリーンショット: 下の画像は Step 8 まで書き終えた完成後の画面です。赤枠の中がこの Step で足した2つの円グラフです。この Step の時点では横に並ばず、縦に積まれた形で出ます。横並びにするのは Step 8 です。

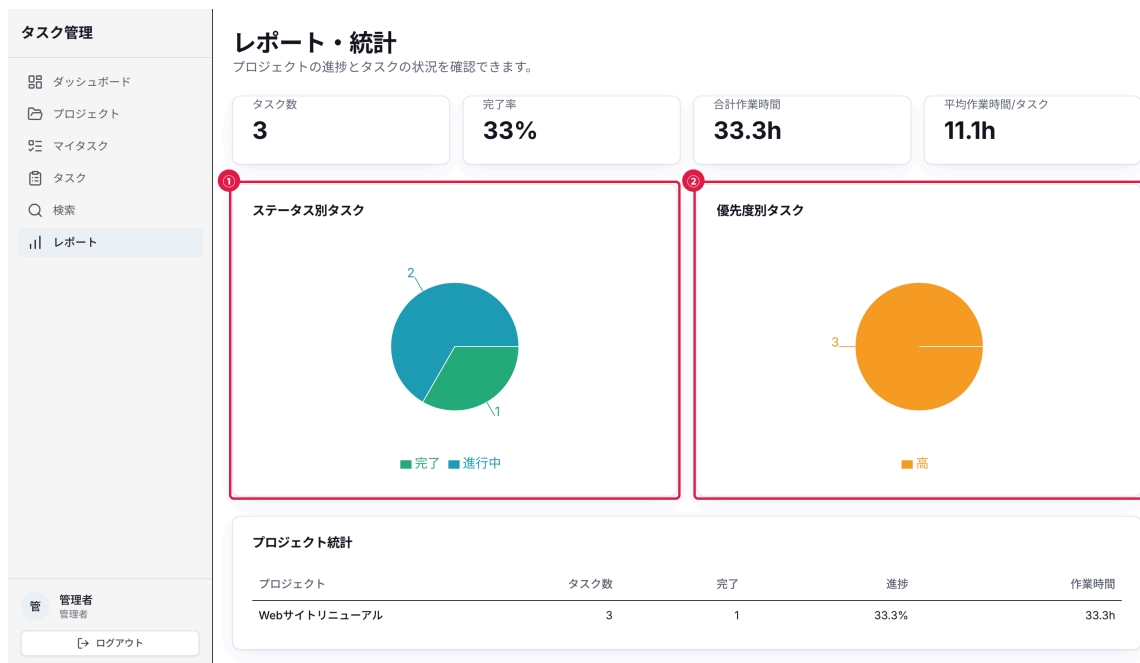


図 4: 完成後のレポートページ。赤枠①がステータス別タスクの円グラフ、赤枠②が優先度別タスクの円グラフ

写っている扇の数値は Day 15 と Day 16 で自分が動かした結果です。ステータスは「進行中」2件と「完了」1件で2切れ、優先度は3件とも「高」なので1切れになります。触った内容が違えば切れの数も変わりますが実装の誤りではありません。優先度の円が1色で埋まるのは `TASK_PRIORITY_COLORS` に4色あってもその色を使う件数が0なら扇は出ないためです。

Step 8: グリッドに配置して完成(読む目安: 4分)

ゴール: 2つのグラフカードを横並びのグリッドに配置します。

実装: Step 4-5 と Step 7 で作った 2つの `<Card>` を、1つのグリッド用 `<div>` で囲みます。Day 21 の統計カード 4 枚を囲むグリッドの閉じタグ直後、プロジェクト統計テーブルの手前へ外枠を追加します。その開始タグと閉じタグの間へ、2つのグラフカードを移してください。

```
{/* filepath: src/app/report/page.tsx */}
{/* グリッドの外枠(この2行の間に Card を移す) */}
<div className="grid grid-cols-1 md:grid-cols-2 gap-6">
</div>
```

次に Step 4-5 で書いたステータス円グラフを移します。その `<Card>` の開始行から `</Card>` の行までをまるごと切り取り、上の開始タグと閉じタグの間に貼り付けます。

続けて Step 7 で書いた優先度円グラフも同じ要領で移します。`<Card>` から `</Card>` までを切り取り、いま貼り付けたステータス円グラフの直後に置きます。

移し終わると `<div>` の中に `<Card>` が2つ、上から「ステータス」「優先度」の順で並びます。

`grid-cols-1 md:grid-cols-2` によりモバイルでは縦並び、PC では横並びのレスポンシブ配置になります。`md:` は画面幅が中サイズ以上のときだけ効く接頭辞です。

Day 22: グラフを表示しよう

確認ポイント:

- PCでは横並び、モバイルでは縦並び
- Day 21 の統計カードの下に配置されています

グラフのブレイクポイント

画面サイズ	クラス	配置
モバイル	<code>grid-cols-1</code>	縦並び
PC	<code>md:grid-cols-2</code>	横並び

Step 9: 動作確認(読む目安: 3分)

ゴール: グラフ表示の全体を確認します。

開発サーバーが動いていればそのまま使います。止めてあるときだけ次のコマンドで起動します。

```
# filepath: ターミナル(確認用)
# 停止している場合だけ起動する
npm run dev
# http://localhost:3000/report にアクセス
```

1. `/report` にアクセス
2. 統計カード(Day 21)の下にグラフがあります
3. ステータス別の円グラフが表示されます
4. 優先度別の円グラフが表示されます
5. 凡例(Legend)で各項目が確認できます

6. マウスオーバーで Tooltip が表示されます

確認ポイント:

- 色がステータス/優先度に対応しています
- Tooltip で件数が確認できます

スクリーンショット: 統計カード 4 枚の下に円グラフ 2 つがグリッド配置された完成画面です。

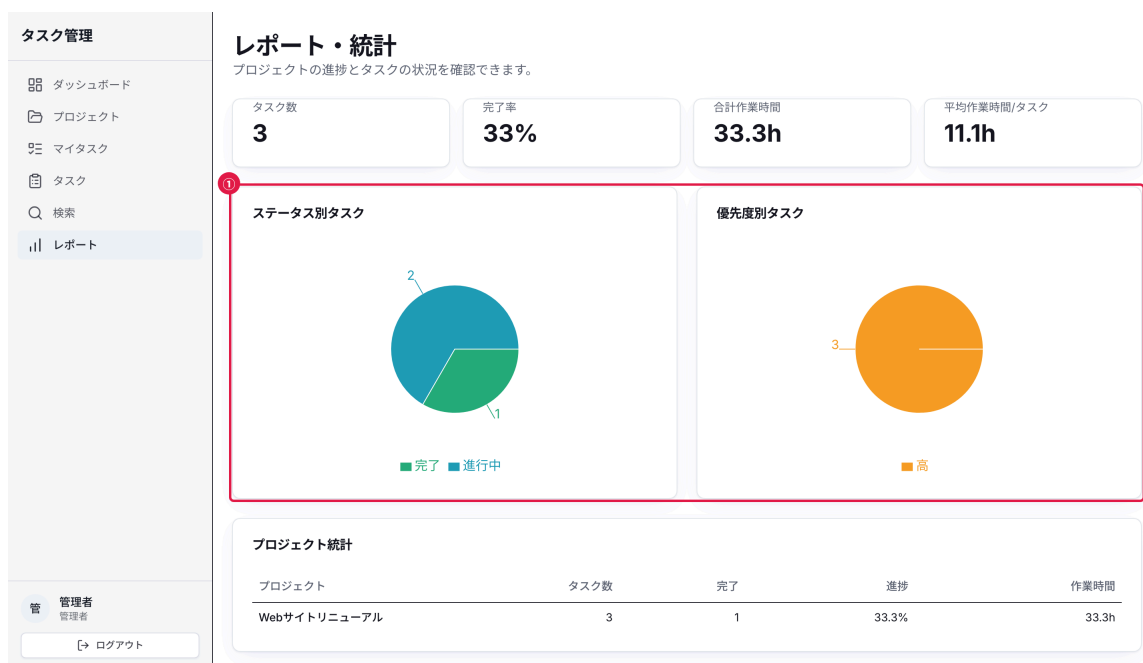


図 5: 完成後のレポートページ。赤枠の中に、2 枚の円グラフが横に並んだグリッドが出ている

Pro パターンで書こう(集計はクライアントかサーバーか)

Before(改善前のコード)

```
// filepath: 読み比べ用サンプル(アンチパターン・実ファイルには対応しません)
// サーバーが集計済みなのに、生の tasks をもう一度数え直す

const todo = tasks.filter((t) => t.status ===
'TODO').length;

const done = tasks.filter((t) => t.status ===
'DONE').length;

const statusData = [
  { key: 'TODO', name: '未対応', value: todo },
  { key: 'DONE', name: '完了', value: done },
];
```

一見すると素直なコードですがこの数行は Day 21 でサーバーが済ませた仕事をやり直しています。しかも **'TODO'** と **'DONE'** を直に書いているので

IN_PROGRESS と **IN_REVIEW** のタスクは円グラフのどこにも現れません。統計カードのタスク数と円グラフの合計が食い違う画面になり、読み手は原因をグラフの描画側で探し始めます。

このコードの問題点:

- サーバーと同じ集計ロジックがフロントにも二重で存在します
- 集計のために全タスクをクライアントへ送る必要があり、通信量が増えます
- ステータスが増えたときサーバーとフロントの両方を直す羽目になります

After(プロが書くコード)

```
// filepath: 読み比べ用サンプル(実ファイルには対応しません)
// サーバー集計済みの statusData を受け取り name だけ足す
const statusData =
  overview?.statusData.map((entry) => ({
    ...entry,
    name: isTaskStatus(entry.key)
      ? TASK_STATUS_LABELS[entry.key]
      : entry.key,
  }))) ?? [];
```

書き換えた後のフロントは受け取った配列に `name` を1つ足すだけになりました。ステータスの種類が増えてもこの行は触らずに済み、サーバーの集計に新しい `key` が加われば扇がそのまま1枚増えます。末尾の `?? []` があるのでタスクが1件も無いアカウントでも `statusData` は空配列になり、扇が描かれないだけでページは落ちません。Day 09 の一覧画面で `?? 0` を使って件数を守ったのと、狙いは変わりません。

このコードの強み:

- 集計はサーバーの1か所だけです。ロジックが二重になりません
- 送るデータは数行の集計結果だけで済みます
- `statusData` の型は `tRPC` が推論するので同じ形の型を手書きしません

覚えておきたいエッセンス

集計はデータの近くにあるサーバー側で1回だけ行い、クライアントは受け取った結果をそのまま描きます。型は tRPC が推論するので同じ形の型を手書きで二重に持たないよう気をつけます。

完成コード全体

今日触ったファイルは1つだけです。ただし Day 21 で書いたコードの上へ、インポート・定数・データ加工・JSX を別々の場所に貼り重ねました。どこへ何を入れたか分からなくなった場合は以下のコードで `src/app/report/page.tsx` を丸ごと置き換えてください。上から順に読むと Step 2 から Step 8 で書いた断片が1つのファイルへどう収まったかを確認られます。

ファイル	役割	対応する Step
<code>src/app/report/page.tsx</code>	レポート画面。統計カード・円グラフ2枚・プロジェクト統計テーブルを1ページに並べる	Step 2～Step 8

`src/app/report/page.tsx`

画面部品の取り込み:

```
// filepath: src/app/report/page.tsx
// 完成版: 画面部品の取り込み
'use client';

import {
  Cell, Legend, Pie, PieChart,
  ResponsiveContainer, Tooltip,
} from 'recharts';
import { AppLayout } from '@component/layout/app-layout';
import {
  Card, CardContent,
  CardHeader, CardTitle,
} from '@component/ui/card';
import { PageLoadingSpinner } from
  '@component/ui/loading-spinner';
import {
  Table, TableBody, TableCell,
  TableHead, TableHeader, TableRow,
} from '@component/ui/table';
```

`recharts` が先頭に来て `@/` で始まる自分のプロジェクトの部品が後ろに続くのは Biome が外部ライブラリを先に並べる決まりだからです。手元の並びが違ってても `npm run fix` で同じ形に直るので写経のときに順番を気にする必要はありません。並びより大事なのは中身のほうで、`recharts` の6つのうち1つでも欠けるとその部品を書いた行で名前が見つからないというエラーになります。

色とラベルの定数:

Day 22: グラフを表示しよう

```
// filepath: src/app/report/page.tsx(同じファイルの続き)
// 完成版: 色とラベルの定数、通信の入口
import {
  isTaskPriority,
  TASK_PRIORITY_COLORS,
  TASK_PRIORITY_LABELS,
} from '@lib/constant/priority';
import {
  isTaskStatus,
  TASK_STATUS_COLORS,
  TASK_STATUS_LABELS,
} from '@lib/constant/status';
import { api } from '@trpc/react';

const CHART_FALLBACK_COLOR = '#9e9e9e';
```

優先度とステータスで、型ガード・色・ラベルの3点セットをそろえて取り込みます。色とラベルを画面側に書き写さないのはあとで定数を直したときにこのページだけ古い色で取り残されるのを防ぐためです。`CHART_FALLBACK_COLOR` はファイルの一番外側に置きます。コンポーネント関数の中へ入れても動きますが描き直しのたびに同じ文字列を作り直すことになります。

取得と表示用の値づくり:

```
// filepath: src/app/report/page.tsx
// 完成版: 取得と表示用の値づくり
export default function ReportPage() {
  const { data: overview, isLoading } =
    api.report.getOverview.useQuery();

  const totalTasks = overview?.totalTasks ?? 0;
  const completionRate =
    overview?.completionRate ?? 0;
  const totalTimeHours =
    ((overview?.totalTimeSpent ?? 0) / 60)
      .toFixed(1);
  const averageTimeHours =
    ((overview?.averageTimePerTask ?? 0) / 60)
      .toFixed(1);
}
```

サーバーから届く数字を、画面へ出す形に整えるのがこの4行です。時間の2つだけ `/ 60` を挟んでいるのはサーバーが分で返すからです。割り算を JSX の中へ書かず先に済ませておくと表示部分は変数名を置くだけになり、カードのどれが何の値かを目で追えます。`?? 0` は `undefined / 60` が `NaN` になるのを防ぐ守りです。

円グラフへ渡す2つの配列:

```
// filepath: src/app/report/page.tsx(同じファイルの続き)
// 完成版: 円グラフへ渡す2つの配列

const statusData =
  overview?.statusData.map((entry) => ({
    ...entry,
    name: isTaskStatus(entry.key)
      ? TASK_STATUS_LABELS[entry.key]
      : entry.key,
  }))) ?? [];

const priorityData =
  overview?.priorityData.map((entry) => ({
    ...entry,
    name: isTaskPriority(entry.key)
      ? TASK_PRIORITY_LABELS[entry.key]
      : entry.key,
  }))) ?? [];

if (isLoading) {
  return <PageLoadingSpinner />;
}
```

2つの配列は読み込み判定より前に置きます。`if (isLoading)`の下へ移しても画面は動きますが`return`を挟んだ先で`const`を書くことになり、値を用意する場所が読み込み判定の前後に散らばります。`?? []`があるのでまだ何も届いていない一瞬でも`statusData`は配列のままです。型を書いていないのはtRPCが`getOverview`の戻り値から`key`と`value`の形を推論するからです。

return の外枠と見出し:

```
// filepath: src/app/report/page.tsx
// 完成版: return の外枠と見出し
return (
  <AppLayout>
    <div className="space-y-6">
      <div>
        <h1 className="text-3xl
          font-bold tracking-tight">
          レポート・統計
        </h1>
        <p className="text-muted-foreground">
          プロジェクトの進捗とタスクの状況を確認できます。
        </p>
      </div>
    </div>
  </AppLayout>
)
```

AppLayout で囲んでいるのでサイドバーとヘッダーはこのファイルに書かなくても付いてきます。**space-y-6** を持つ **div** がこのあと並べる統計カード・グラフ・テーブルの間隔をまとめて決めます。カードごとに余白を書かずに済むのは縦に積む要素の間隔を親が1か所で持っているからです。

統計カードの前半:

Day 22: グラフを表示しよう

```
    { /* filepath: src/app/report/page.tsx(同じファイ  
ルの続き) */}  
    { /* 完成版: 統計カード(タスク数・完了率) */}  
    <div className="grid grid-cols-1  
      sm:grid-cols-2 lg:grid-cols-4 gap-4">  
      <Card>  
        <CardContent className="pt-6">  
          <p className="text-sm  
            text-muted-foreground mb-1">  
            タスク数</p>  
          <p className="text-3xl font-bold">  
            {totalTasks}</p>  
        </CardContent>  
      </Card>  
      <Card>  
        <CardContent className="pt-6">  
          <p className="text-sm  
            text-muted-foreground mb-1">  
            完了率</p>  
          <p className="text-3xl font-bold">  
            {completionRate}%</p>  
        </CardContent>  
      </Card>
```

グリッドの列数を画面幅で3段階に変えているのは `text-3xl` の数字が折り返さない幅を保つためです。4枚を横一列に並べたまま画面を狭めると1枚あたりの幅が数字の桁数を下回り、`120` のような値が2行に割れます。カードの中身が2枚とも同じ形なのは見出しの文字と数字の大きさをそろえて4枚を1つのまとまりとして読ませるためです。

統計カードの後半:

Day 22: グラフを表示しよう

```
    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }

    { /* 完成版: 統計カード(作業時間の合計と平均) */ }

    <Card>
      <CardContent className="pt-6">
        <p className="text-sm text-muted-foreground mb-1">
          合計作業時間</p>
        <p className="text-3xl font-bold">
          {totalTimeHours}h</p>
      </CardContent>
    </Card>
    <Card>
      <CardContent className="pt-6">
        <p className="text-sm text-muted-foreground mb-1">
          平均作業時間/タスク</p>
        <p className="text-3xl font-bold">
          {averageTimeHours}h</p>
      </CardContent>
    </Card>
  </div>
```

時間の2枚は変数の中身がすでに文字列なので `h` を後ろに足すだけです。末尾の `</div>` は前のブロックで開いたグリッドを閉じるタグになります。ここを書き忘れると後続の閉じタグとの対応がずれ、JSXの構文エラーになります。

ステータス円グラフの枠:

```
    { /* filepath: src/app/report/page.tsx(同じファイ  
ルの続き) */}  
    { /* 完成版: ステータス円グラフの枠 */}  
    <div className="grid grid-cols-1  
      md:grid-cols-2 gap-6">  
      <Card>  
        <CardHeader>  
          <CardTitle>ステータス別タスク</CardTitle>  
        </CardHeader>  
        <CardContent>  
          <div className="h-[300px]">  
            <ResponsiveContainer  
              width="100%" height="100%">  
              <PieChart>  
                <Pie  
                  data={statusData}  
                  dataKey="value"  
                  nameKey="name"  
                  cx="50%"  
                  cy="50%"  
                  outerRadius={80}  
                  label  
                >
```

Day 22: グラフを表示しよう

`h-[300px]` を持つ `div` が `ResponsiveContainer` の `height="100%"` の基準になります。この `div` を外すと基準の高さが決まらず、100% は 0 と計算されて扇が1枚も描かれません。グラフのグリッドを統計カードとは別の `div` にしてあるのは、カードは4列、グラフは2列と、ちょうどよい列数が違うからです。

ステータス円グラフの色付け:

```

        { /* filepath:
src/app/report/page.tsx(同じファイルの続き) */ }
        { /* 完成版: ステータス円グラフの色と
閉じタグ */ }

        {statusData.map((entry) => (
          <Cell
            key={entry.key}
            fill={
              isTaskStatus(entry.key)
                ? TASK_STATUS_COLORS[
entry.key]
                : CHART_FALLBACK_COLOR
            }
          />
        ))}
      </Pie>
      <Tooltip />
      <Legend />
    </PieChart>
  </ResponsiveContainer>
</div>
</CardContent>
</Card>

```

`Cell` はデータ1件につき1つ作り、その扇の色だけを受け持ちます。`entry.key` の型はステータスの5つの文字列に決まっているため、型ガードを外しても型エラーにはなりません。`isTaskStatus` を挟むのは、実行中に対応表へ無い値が渡った

Day 22: グラフを表示しよう

場合に `undefined` ではなく代替色を選ぶためです。`Tooltip` と `Legend` を `Pie` の外に置くのはこの2つが扇そのものではなくグラフ全体に付く表示だからです。

優先度円グラフの枠:

```
    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }
    { /* 完成版: 優先度円グラフの枠 */ }
    <Card>
      <CardHeader>
        <CardTitle>優先度別タスク</CardTitle>
      </CardHeader>
      <CardContent>
        <div className="h-[300px]">
          <ResponsiveContainer
            width="100%" height="100%">
            <PieChart>
              <Pie
                data={priorityData}
                dataKey="value"
                nameKey="name"
                cx="50%"
                cy="50%"
                outerRadius={80}
                label
              >
```

ステータス側と変えたのは `CardTitle` の文言と `data` に渡す配列だけです。

`dataKey` と `nameKey` を書き換えずに済むのは Step 3 と Step 6 で2つの配列を `value` と `name` という同じプロパティ名にそろえたからです。形をそろえておくとグラフを増やすたびに設定を読み直す手間が減ります。

優先度円グラフの色付け:

```
        { /* filepath:
src/app/report/page.tsx(同じファイルの続き) */ }

        { /* 完成版: 優先度円グラフの色と閉じ
タグ */ }

        {priorityData.map((entry) => (
            <Cell
                key={entry.key}
                fill={
                    isTaskPriority(entry.key)
                        ? TASK_PRIORITY_COLORS[
entry.key]
                        : CHART_FALLBACK_COLOR
                }
            />
        ))}
    </Pie>
    <Tooltip />
    <Legend />
</PieChart>
</ResponsiveContainer>
</div>
</CardContent>
</Card>
</div>
```

最後の `</div>` でグラフ用のグリッドを閉じます。ここまでで `<Card>` が2つ入った状態になり、画面幅が `md` 以上なら横並び、それより狭ければ縦積みになります。閉じ忘れると後続の閉じタグとの対応がずれ、JSXの構文エラーになります。

プロジェクト統計テーブルの見出し:

```
    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }
    { /* 完成版: プロジェクト統計テーブルの見出し */ }
    <Card>
      <CardHeader>
        <CardTitle>プロジェクト統計</CardTitle>
      </CardHeader>
      <CardContent>
        <Table>
          <TableHeader>
            <TableRow>
              <TableHead className="w-[200px]">
                プロジェクト</TableHead>
              <TableHead className="text-right">
                タスク数</TableHead>
              <TableHead className="text-right">
                完了</TableHead>
              <TableHead className="text-right">
                進捗</TableHead>
              <TableHead className="text-right">
                作業時間</TableHead>
            </TableRow>
          </TableHeader>
```

Day 22: グラフを表示しよう

数字の列に `text-right` を付けているのは桁数の違う値を右端でそろえるためです。`7` と `123` を左寄せで並べると一の位の位置がずれ、大小を見比べるのに時間がかかります。先頭の列だけ `w-[200px]` で幅を固定してあるのはプロジェクト名の長さで右側の列がずれないようにするためです。

プロジェクト統計テーブルの行:

```
    { /* filepath: src/app/report/page.tsx(同じ
ファイルの続き) */}

    { /* 完成版: プロジェクト統計テーブルの行 */}

    <TableBody>
      {overview?.projectStats.map((stat) => (
        <TableRow key={stat.id}>
          <TableCell className="font-medium">
            {stat.name}</TableCell>
          <TableCell className="text-right">
            {stat.totalTasks}</TableCell>
          <TableCell className="text-right">
            {stat.completedTasks}</TableCell>
          <TableCell className="text-right">
            {stat.progress.toFixed(
1)}}%</TableCell>
          <TableCell className="text-right">
            {stat.totalTimeHours.toFixed(1)}h
          </TableCell>
        </TableRow>
      ))}
    </TableBody>
```

`key` に `stat.id` を渡すのは React が描き直しのときにどの行がどのプロジェクトかを見分けるためです。配列の番号を渡すとプロジェクトを新しく作って並び順が変わったときに前の行の表示を別のプロジェクトへ流用します。`toFixed(1)` は表示のときだけ小数を 1 桁へ丸める指定で、サーバーが返す `71.42857142857143` をそのまま出さないための処理です。

ファイル末尾の閉じタグ:

```
        { /* filepath: src/app/report/page.tsx(同じフ
ファイルの続き) */}
        { /* 完成版: 閉じタグと関数の終わり */}
      </Table>
    </CardContent>
  </Card>
</div>
</AppLayout>
);
}
```

開いたタグを内側から順に閉じてファイルが終わります。`Table` から `AppLayout` まで開いた順の逆にたどるのが JSX の決まりです。順番を入れ違えると保存した瞬間に構文エラーが出ます。ブラウザに映るのはグラフではなくエラー画面です。画面が真っ白になったときはまずこの閉じタグの並びを上から数えてください。

今日のまとめ

- ☐ Recharts で円グラフを表示できました
- ☐ `getOverview` の集計済みデータを受け取って表示しました
- ☐ `key` から色とラベルを引いて色分けしました

- ☐ レスポンシブに2列配置できました

つまずきポイント

エラー / 問題	原因	解決方法
グラフが表示されない	親に高さが無い	<code>h-[300px]</code> を親に設定
グラフが空になる	<code>dataKey</code> / <code>nameKey</code> がデータの形と不一致	<code>value</code> と <code>name</code> のプロパティ名を合わせる
全部同じ色になる	Cell 未使用	map で Cell に色を設定
凡例が表示されない	Legend 未追加	PieChart 内に Legend 追加
サイズが固定される	ResponsiveContainer 未使用	親に <code>h-[300px]</code> 、 <code>ResponsiveContainer</code> に幅・高さ100%を指定

今日学んだ用語

用語	意味
PieChart	円グラフの枠組みコンポーネント
Cell	円グラフの各スライスに色を付ける
ResponsiveContainer	親のサイズに合わせる自動調整コンテナ
dataKey	扇の大きさに使うプロパティ名の指定
nameKey	凡例やツールチップの見出しに使うプロパティ名の指定

用語	意味
getOverview	集計済みのレポートデータを返す API

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `statusData` を作る `map` はサーバーから届いた配列に何を1つ足していますか。

A. `name`、つまり日本語のラベルだけです。件数の `value` はサーバーが数え終わっている所以画面側では数え直しません。`entry.key` の `'TODO'` を `TASK_STATUS_LABELS` に通して `'未対応'` を得ています。

Q2. `<div className="h-[300px]">` を外して `ResponsiveContainer` を `CardContent` の直下に置くと画面はどうなりますか。

A. 扇が1枚も描かれず、カードの中は空のままになります。

`ResponsiveContainer` の `height="100%"` は親を基準にした割合なので親に高さが無いと100%が0と計算されるためです。

Q3. `TASK_STATUS_COLORS[entry.key]` を引く前に `isTaskStatus(entry.key)` を通すのはなぜですか。`as` で押し込まないのはなぜですか。

A. 型エラーを避けるためではありません。`entry.key` の型は Prisma の enum と同じ 5 つの文字列なので、型ガード無しでも型検査は通ります。型ガードを置くのは、実行中に対応表へ無い値が渡った場合に `CHART_FALLBACK_COLOR` へ分岐するためです。`as` は実行時の値を確かめないため、この分岐の代わりにはなりません。

追加課題:円グラフの大きさだけを変える

件数を変えずに見た目を調整します。理解チェック Q1 の `value` と、円の大きさを決める設定を区別しましょう。

前提はステータス円グラフに 1 つ以上の扇が表示されることです。表示中のラベルと件数を控えておきます。

`src/app/report/page.tsx` で `statusData` を渡している `Pie` を探し、`outerRadius` を 80 から 60 へ変えてください。優先度グラフと、親の高さを指定する要素は変更しません。

画面を再読み込みし、ステータスの円だけが小さくなったことを確認します。ラベルと件数が変更前と一致すれば成功です。`dataKey` を変えずに表示を調整できた理由を、Step 5 の説明から答えてみましょう。

円が消えた場合は理解チェック Q2 の親の高さと、`statusData` を渡す箇所を確認します。確認後は `outerRadius` を 80 へ戻してください。DB のデータは変更しません。

次回予告

Day 23 では週次レポート機能を実装します。週ごとの完了タスク数を棒グラフと折れ線グラフで確認します。

次に読むもの

- 前の日: Day 21^{*1}
- 次の日: Day 23^{*2}
- 全体の地図: 学びのロードマップ^{*3}
- 目次: カリキュラム目次^{*4}
- 詰まったとき: トラブルシューティング^{*5}
- 言葉の意味: 用語集^{*6}

奥付

タイトル	Day 22: グラフを表示しよう
著者	磯貝光佑

1. <https://drive.google.com/file/d/1L0tDk42OVnMY5tsaTfXgfXfLao1j5Sy8/view?usp=drivesdk>
2. https://drive.google.com/file/d/1EaYyLRG4W8EmUULkkja5L_xFsfQJaSGc/view?usp=drivesdk
3. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk
4. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>
5. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>
6. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。